

```
import numpy as np
import pandas as pd

from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# -----
# 1. Load PhiUSIIL Dataset
# -----

dataset = fetch_ucirepo(id=967)

X = dataset.data.features
y = dataset.data.targets

# -----
# 2. Convert Target to Series
# -----
```

```
y = y.iloc[:, 0]
```

```
# -----
```

```
# 3. Remove Unnecessary Columns
```

```
# -----
```

```
remove_columns = ["FILENAME"]
```

```
for column in remove_columns:
```

```
    if column in X.columns:
```

```
        X = X.drop(columns=[column])
```

```
# -----
```

```
# 4. Remove URL and Other Text Columns
```

```
# -----
```

```
categorical_columns = X.select_dtypes(
```

```
    include=["object"]
```

```
).columns
```

```
X = X.drop(columns=categorical_columns)
```

```
# -----
```

```
# 5. Convert Features to Numeric
```

```
# -----
```

```
X = X.apply(
```

```
pd.to_numeric,  
errors="coerce"  
)
```

```
X = X.fillna(0)
```

```
# -----
```

```
# 6. Train-Test Split
```

```
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

```
# -----
```

```
# 7. Feature Scaling
```

```
# -----
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
#-----
```

```
# 8. Build Deep Neural Network
```

```
#-----
```

```
model = Sequential([
```

```
    Dense(
```

```
        128,
```

```
        activation="relu",
```

```
        input_shape=(X_train.shape[1],)
```

```
    ),
```

```
    Dropout(0.30),
```

```
    Dense(
```

```
        64,
```

```
        activation="relu"
```

```
    ),
```

```
    Dropout(0.20),
```

```
    Dense(
```

```
        32,
```

```
        activation="relu"
```

```
    ),
```

```
    Dense(
```

```
        1,
```

```
        activation="sigmoid"
    )
])

# -----

# 9. Compile Model

# -----

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# -----

# 10. Display Model

# -----

model.summary()

# -----

# 11. Train Model

# -----

history = model.fit(
    X_train,
    y_train,
    epochs=10,
```

```
    batch_size=256,
    validation_split=0.2
)

# -----
# 12. Predict Test Data
# -----

probability = model.predict(X_test)

y_pred = (
    probability >= 0.5
).astype(int)

# -----
# 13. Calculate Performance
# -----

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred
)
```

```
recall = recall_score(  
    y_test,  
    y_pred  
)
```

```
f1 = f1_score(  
    y_test,  
    y_pred  
)
```

```
# -----  
# 14. Display Results  
# -----
```

```
print("\nMODEL PERFORMANCE")
```

```
print("Accuracy :", accuracy)  
print("Precision :", precision)  
print("Recall   :", recall)  
print("F1 Score :", f1)
```

```
(shared-ai-venv) PS E:\Ruby-Projects\Sentiment_Analysis_LSTM> python .\hi.py
E:\Ruby-Projects\Sentiment_Analysis_LSTM\hi.py:20: FutureWarning: For backward compatibility, str dtypes are included by select_dtypes when object dtype is specified. This behavior is deprecated and will be
future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
categorical_columns = X.select_dtypes(
E:\Ruby-Projects\shared-ai-venv\Lib\site-packages\keras\src\layers\core\dense.py:103: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `In
ject as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1790151252.310822 14640 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	6,528
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

```
Total params: 16,897 (66.00 KB)
Trainable params: 16,897 (66.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
590/590 4s 5ms/step - accuracy: 0.9943 - loss: 0.0206 - val_accuracy: 0.9997 - val_loss: 0.0011
Epoch 2/10
590/590 2s 3ms/step - accuracy: 0.9998 - loss: 8.7194e-04 - val_accuracy: 0.9998 - val_loss: 8.0784e-04
Epoch 3/10
590/590 2s 3ms/step - accuracy: 0.9998 - loss: 6.8322e-04 - val_accuracy: 0.9998 - val_loss: 7.7037e-04
Epoch 4/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 5.9832e-04 - val_accuracy: 0.9999 - val_loss: 4.7508e-04
Epoch 5/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 2.9112e-04 - val_accuracy: 0.9999 - val_loss: 5.4816e-04
Epoch 6/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 3.5383e-04 - val_accuracy: 0.9999 - val_loss: 5.5847e-04
Epoch 7/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 2.3319e-04 - val_accuracy: 0.9999 - val_loss: 7.0695e-04
Epoch 8/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 2.3442e-04 - val_accuracy: 0.9999 - val_loss: 9.4249e-04
Epoch 9/10
590/590 2s 3ms/step - accuracy: 0.9999 - loss: 3.3828e-04 - val_accuracy: 0.9999 - val_loss: 8.5337e-04
Epoch 10/10
590/590 2s 3ms/step - accuracy: 1.0000 - loss: 1.3390e-04 - val_accuracy: 0.9999 - val_loss: 7.9300e-04
1474/1474 1s 889us/step

MODEL PERFORMANCE
Accuracy : 0.9999575902796921
Precision : 0.9999258490286222
Recall : 1.0
F1 Score : 0.9999629231396685
(shared-ai-venv) PS E:\Ruby-Projects\Sentiment_Analysis_LSTM>
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
# -----
# 1. Load RT-IoT2022 Dataset
# -----
```

```
data = pd.read_csv("RT_IOT2022.csv")

print("Dataset Shape:", data.shape)
print(data.head())
```

```
# -----
# 2. Remove Unnecessary Columns
# -----
```

```
remove_columns = [
    "id",
    "Unnamed: 0"
]

for column in remove_columns:
    if column in data.columns:
        data = data.drop(columns=[column])
```

```
# -----
# 3. Select Target Column
# -----
```

```
target_column = "Attack_type"

X = data.drop(columns=[target_column])
y = data[target_column]
```

```
# -----
# 4. Encode Categorical Features
# -----
```

```
categorical_columns = X.select_dtypes(
    include=["object"])
```

```
).columns
```

```
for column in categorical_columns:
```

```
    encoder = LabelEncoder()
```

```
    X[column] = encoder.fit_transform(  
        X[column].astype(str)  
    )
```

```
# -----  
# 5. Convert Features to Numeric  
# -----
```

```
X = X.apply(  
    pd.to_numeric,  
    errors="coerce"  
)
```

```
X = X.fillna(0)
```

```
# -----  
# 6. Encode Target Classes  
# -----
```

```
target_encoder = LabelEncoder()
```

```
y = target_encoder.fit_transform(y)
```

```
# -----  
# 7. Train-Test Split  
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

```
# -----  
# 8. Feature Scaling  
# -----
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# -----  
# 9. Number of Classes  
# -----
```

```
num_classes = len(  
    np.unique(y_train)  
)
```

```
# -----  
# 10. Build DNN  
# -----
```

```
model = Sequential([  
    Dense(  
        128,  
        activation="relu",  
        input_shape=(X_train.shape[1],)  
    ),  
    Dropout(0.30),  
    Dense(  
        64,  
        activation="relu"  
    ),  
    Dropout(0.20),  
    Dense(  
        32,  
        activation="relu"  
    ),  
    Dense(  
        num_classes,  
        activation="softmax"  
    )  
)
```

```
# -----  
# 11. Compile Model  
# -----
```

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
# -----  
# 12. Display Model  
# -----
```

```
model.summary()
```

```
# -----  
# 13. Train Model  
# -----
```

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=10,  
    batch_size=256,  
    validation_split=0.2  
)
```

```
# -----  
# 14. Prediction  
# -----
```

```
probabilities = model.predict(X_test)
```

```
y_pred = np.argmax(  
    probabilities,  
    axis=1  
)
```

```
# -----  
# 15. Performance  
# -----
```

```
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)
```

```
precision = precision_score(  
    y_test,  
    y_pred,  
    average="weighted",  
    zero_division=0  
)
```

```
recall = recall_score(  
    y_test,  
    y_pred,  
    average="weighted",  
    zero_division=0  
)
```

```
)

f1 = f1_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)

print("\nMODEL PERFORMANCE")
```

```
print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
=====
MODEL ACCURACY
=====
Accuracy: 0.9974
Accuracy: 99.74%

=====
CLASSIFICATION REPORT
=====
```

	precision	recall	f1-score	support
ARP_poisoning	0.99	0.99	0.99	1550
DDOS_slowloris	1.00	1.00	1.00	107
DOS_SYN_Hping	1.00	1.00	1.00	6726
MQTT_Publish	1.00	1.00	1.00	829
Thing_Speak	0.99	0.99	0.99	1622
Wipro_bulb	0.98	0.94	0.96	51
accuracy			1.00	10885
macro avg	0.99	0.99	0.99	10885
weighted avg	1.00	1.00	1.00	10885

```
=====
CONFUSION MATRIX
=====
```

```
[[1538    0    0    0   12    0]
 [    0  107    0    0    0    0]
 [    0    0 6726    0    0    0]
 [    1    0    0  828    0    0]
 [   11    0    0    0 1610    1]
 [    1    0    0    0    2   48]]
```

